PLANTDISEASEAI

COMPREHENSIVE TECHNICAL PROJECT DOCUMENT

Grape & Tomato Plant Leaf Disease Detection

EfficientNet-B0 | Grad-CAM | PySide6 | Multilingual Guidance

Technical + Academic Project Documentation

Prepared from the current repository, retained project reports, evaluation artifacts, and verified application behavior

Document status: Evidence-grounded technical edition
Project version in configuration: 1.0.0
Prepared: 25 August 2026

Accuracy convention
Historical training and evaluation metrics are labelled as reported results. The Windows executable, two-model loading, synthetic-image inference, and Grad-CAM generation are separately identified as independently verified packaging tests.

Document Control and Evidence Classification

This edition is a new technical document rather than a transcription of the supplied 26-page report. It reconciles that report with the current repository after cleanup and application packaging. Statements are classified to avoid presenting historical claims as fresh experimental verification.

Table 1. Evidence labels used throughout this document

Label	Meaning	Examples
Current implementation	Directly present in current source, configuration, models, or application delivery	EfficientNet-B0 factory, Grad-CAM code, two final .pth files, multilingual app
Reported historical result	Persisted in retained reports or outputs but not rerun for this document	Grape 100% on 610; tomato 99.8526% on 2,713
Independently verified	Executed during current repository preparation	Windows EXE start, both model loads, synthetic inference, Grad-CAM overlays
Documented pathway	Described in historical documentation/configuration; complete deployable artifact not retained	ONNX/Android pathway and Raspberry Pi workflow
Not available	Required evidence is absent from the current repository	Raw images, current ONNX exports, Android project, user study

Repository snapshot

The current root contains README.md, requirements.txt, Dataset/, Models/, Evaluation/, Outputs/, Documentation/, and App/. Raw PlantVillage images and prepared split directories are intentionally absent.

Executive Summary

PlantDiseaseAI is a crop-selectable desktop system for classifying grape and tomato leaf images. The retained production path uses two crop-specific EfficientNet-B0 checkpoints, deterministic ImageNet-style preprocessing, softmax Top-3 prediction, and native Grad-CAM visualization. A PySide6 interface provides image upload, camera integration, crop switching, and English, Hindi, and Marathi localization. Optional Groq integration can request structured guidance when a user supplies an API key and network connectivity is available.

The current repository retains final PyTorch weights for four grape classes and ten tomato classes. Reported held-out PlantVillage results are 100.00% accuracy on 610 grape images and 99.8526% accuracy on 2,713 tomato images. These figures describe curated held-out data, not field performance. Retained runtime diagnosis also demonstrates overconfident tomato predictions for out-of-distribution inputs, including grape and synthetic images. Accordingly, this project should be treated as decision support and an academic engineering demonstration rather than a validated agronomic diagnostic instrument.

The Windows application was consolidated into a single editable app.py and packaged as a 398.41 MiB one-file executable. During the current preparation work, the EXE launched successfully on 64-bit Windows 11, loaded both final checkpoints, completed synthetic-image inference for both crops, and produced Grad-CAM overlays. Camera hardware, live Groq calls, Raspberry Pi deployment, and Windows versions other than Windows 11 were not independently verified.

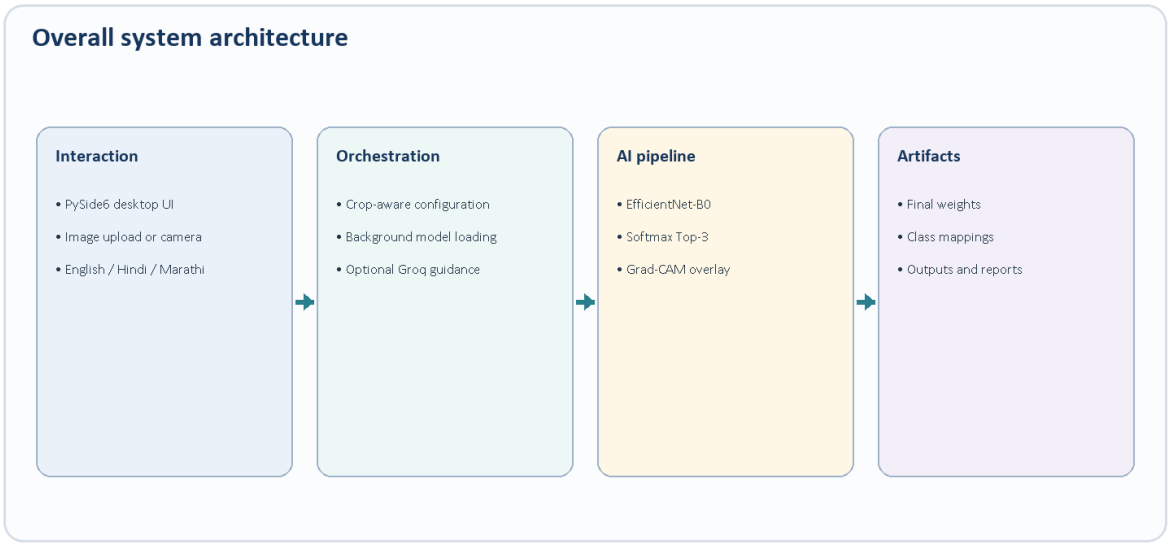

Figure 1. Current PlantDiseaseAI logical architecture.

Abstract

Automated analysis of leaf imagery can provide rapid preliminary screening where expert inspection is delayed or unavailable. This project implements a two-crop plant leaf classifier using ImageNet-pretrained EfficientNet-B0 networks and crop-specific output heads. Grape inference covers Black Rot, Esca (Black Measles), Leaf Blight (Isariopsis Leaf Spot), and Healthy. Tomato inference covers ten bacterial, fungal, viral, mite-related, and healthy classes. Images are converted to RGB, resized to 224 x 224 for grape or 256 x 256 for tomato, normalized using ImageNet statistics, and passed through the selected checkpoint. Softmax probabilities provide ranked predictions, while Grad-CAM produces a spatial explanation for the top class.

The surrounding software converts the model pipeline into a desktop application. PySide6 provides the graphical interface, OpenCV provides image and camera handling, background Qt workers reduce UI blocking, and embedded translation catalogs support English, Hindi, and Marathi. The final Windows executable bundles the Python runtime and application dependencies but deliberately reads model weights, YAML configuration, and class mappings from their standard repository folders to avoid unnecessary duplication.

Results retained in the repository show excellent performance on PlantVillage-style held-out test sets; however, dataset homogeneity, absence of field validation, and demonstrated out-of-distribution overconfidence constrain interpretation. The contribution is therefore the integration of data-quality controls, transfer learning, explainability, localization, application engineering, and packaging into an evidence-documented project, not a claim of universal real-world diagnostic accuracy.

- Keywords: plant disease classification, EfficientNet-B0, transfer learning, Grad-CAM, PySide6, PlantVillage, explainable AI, edge deployment.

Table of Contents

The DOCX contains a live Word table-of-contents field. The accompanying PDF contains the rendered heading hierarchy and page sequence.

SECTION 01

1. Project Overview

Motivation, system concept, and verified boundaries

1.1 Motivation

Leaf symptoms can be visually distinctive, but access to trained agricultural expertise is uneven and manual identification is sensitive to observer experience, image quality, and symptom similarity. PlantDiseaseAI explores whether a compact convolutional classifier can provide a fast first-level screening result while also exposing the image regions that influenced its decision.

1.2 Proposed solution

The implemented workflow combines two separately trained EfficientNet-B0 classifiers behind one crop-aware interface. Configuration files determine class order, image resolution, model path, class-mapping path, device preference, and application settings. This avoids mixing grape and tomato outputs and keeps the inference path consistent with the documented evaluation transforms.

1.3 Key contributions

- Two retained crop-specific final checkpoints rather than one ambiguous multi-domain model.
- Native Grad-CAM integrated with the same PyTorch predictor used for classification.
- A multilingual PySide6 application with crop switching, upload, camera abstractions, and optional Groq guidance.
- Evidence-preserving reports for dataset audit, balancing, split integrity, evaluation, misclassifications, and runtime diagnosis.
- A tested one-file Windows executable plus an Inno Setup source script that references final runtime assets without duplicating them inside App/.

SECTION 02

2. Problem Statement

The engineering problem is to transform crop-specific image classifiers into a usable, explainable, and locally executable application while maintaining correct class mapping and preprocessing. The scientific problem is narrower: infer one label from a fixed class set using a single leaf image. The application does not identify arbitrary crops, unknown diseases, nutrient deficiencies, or non-leaf conditions.

Formal problem

Given an image I and a user-selected crop c in {grape, tomato}, estimate a probability distribution p(y | I, c) over the configured class set, return ranked classes, and generate an explanatory activation map for the highest-scoring class when Grad-CAM is enabled.

The project must also handle invalid files, class-count mismatches, unavailable cameras, missing optional API configuration, CPU-only execution, and packaging without requiring end users to install Python. These constraints make the work a software-system project as well as a model-training project.

SECTION 03

3. Objectives

Table 2. Project objectives and current evidence

Objective	Status	Evidence
Classify grape leaf images into four classes	Implemented	Final grape checkpoint, config, class mapping,

Objective	Status	Evidence
		evaluator outputs
Classify tomato leaf images into ten classes	Implemented	Final tomato checkpoint, config, class mapping, evaluator outputs
Provide Grad-CAM explanations	Implemented and packaged-test verified	gradcam.py, explainable predictor, generated overlay tests
Provide multilingual desktop UI	Implemented	English, Hindi, Marathi catalogs embedded in consolidated app.py
Support image upload and camera capture	Implemented	Qt controller and OpenCV/picamera2 camera abstractions embedded in app.py
Distribute a Windows application	Implemented and launch verified	Plant-Disease-AI.exe and Plant-Disease-AI.iss
Support Raspberry Pi	Implemented pathway; not currently hardware-verified	Pi configuration and platform/camera logic
Provide Android/ONNX deployment	Documented pathway only	Historical documentation; no current Android project or ONNX model retained

SECTION 04

4. Scope and Boundaries

4.1 Included

- Single-image classification for grape and tomato leaf imagery.
- Top-1 and Top-3 probability presentation.
- Grad-CAM explanation for the PyTorch inference backend.
- Windows desktop delivery and source execution.
- Optional local camera acquisition and optional network-based Groq guidance.

4.2 Excluded or unavailable

- Raw PlantVillage images and prepared split folders are not published in the current repository.
- Potato is audit-only; no final potato classifier is retained.
- No crop verifier or general out-of-distribution rejection model is implemented.
- No field trial, farmer user study, agronomist validation, treatment-outcome study, or regulatory assessment is available.
- Current ONNX exports and the Android Studio project are not retained.

Responsible scope
A confidence score is not a measure of agronomic certainty. The system should not be used as the sole basis for pesticide selection or high-value crop intervention.

SECTION 05

5. Existing and Traditional Approaches

Traditional identification relies on visual examination of lesion shape, color, distribution, texture, and progression, often combined with knowledge of crop stage, weather, soil, and local disease prevalence. Expert examination can incorporate context unavailable to an image classifier, but access may be delayed and results can vary with expertise.

A model-only notebook improves speed and repeatability but does not solve deployment, localization, input validation, explainability, or failure handling. Cloud-only systems additionally require connectivity and may raise privacy or operational concerns. PlantDiseaseAI addresses part of this gap through local inference, a crop-constrained interface, and explanatory overlays, while retaining the limitations of single-image supervised classification.

Table 3. Approach comparison

Approach	Strength	Limitation
Expert inspection	Context-aware and capable of differential diagnosis	May be unavailable or delayed
Notebook classifier	Useful for model development and reproducibility	Not accessible to non-technical users
Cloud inference	Centralized updates and scalable compute	Connectivity and data-transfer dependency
PlantDiseaseAI	Offline core inference, explainability, multilingual desktop UI	Restricted classes and limited field generalization evidence

SECTION 06

6. Proposed System and User Journey

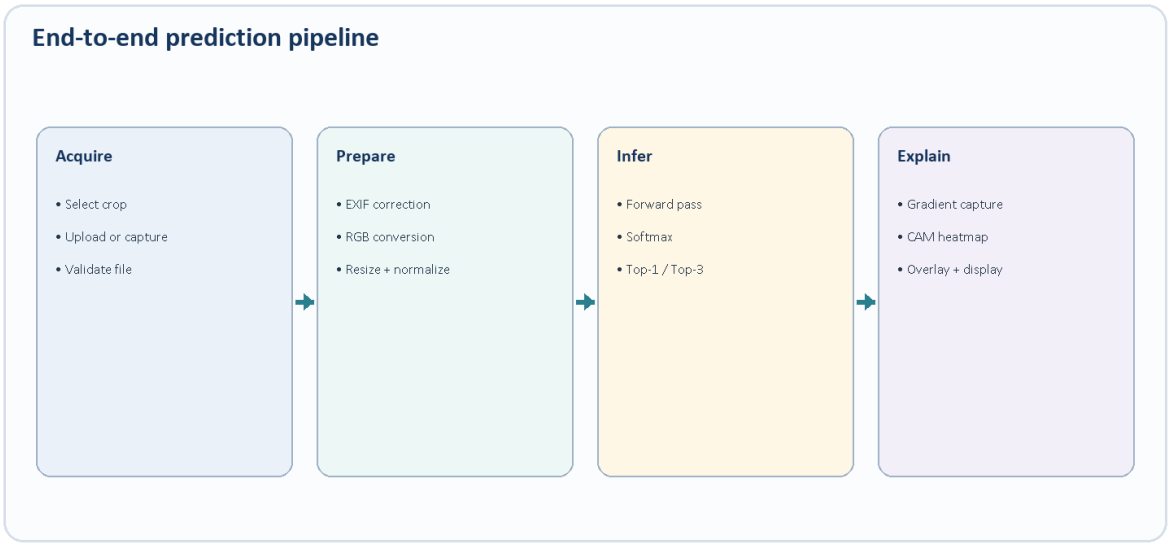

Figure 2. End-to-end crop-specific prediction pipeline.

The user selects the crop before analysis, reducing the output space to the relevant class mapping. An uploaded or captured image is validated and passed to deterministic transforms. The selected EfficientNet-B0 produces logits, softmax converts them into normalized class probabilities, and the interface shows ranked results. The explainable predictor then computes Grad-CAM for the top class without replacing the prediction.

The application can subsequently request guidance through Groq. This stage is optional and logically separated from classification: a missing key or network failure must not invalidate the model result. Current source behavior exposes an unavailable state when Groq cannot be used; no live Groq response was tested for this document.

SECTION 07

7. System Requirements

Table 4. Software and runtime requirements

Area	Current requirement or evidence
Source runtime	Documented Python target: 3.11; requirements list supports PyTorch, torchvision, PySide6, OpenCV, Albumentations, Pillow, NumPy, scikit-learn, PyYAML, imagehash, matplotlib, seaborn, tqdm, python-dotenv, and groq.
Windows delivery	One-file 64-bit Windows executable, independently tested on Windows 11; Python not required for end users.
Runtime assets	Models/Final_Model configs and weights plus grape/tomato class_mapping.json files under Dataset/.
Optional network	Required only for Groq guidance; core classification and Grad-CAM are local.
Camera	OpenCV webcam support; picamera2 is conditionally referenced for Raspberry Pi.

Table 5. Hardware status

Platform	Evidence-based statement
Windows PC	Verified on the current 64-bit Windows 11 machine. CPU inference succeeded for both models.
GPU workstation	CUDA auto-selection is implemented. Historical training reports mention an RTX 4050 for grape; not independently rerun.
Raspberry Pi	Pi-aware CPU/camera/memory settings are implemented and documented. Current hardware test evidence is not available.
Android	No current Android artifact is present. Treat as a documented future/deployment pathway.

SECTION 08

8. Dataset Source and Governance

Crop configuration identifies the dataset source as PlantVillage (Original). The current repository does not publish raw imagery. It retains class mappings, audit reports, split reports, balancing reports, and evaluation outputs. Consequently, dataset counts below are historical recorded results rather than a fresh recount of image directories.

Table 6. Dataset summary

Crop	Raw/configured total	Prepared split evidence	Classes	Current raw images
Grape	4,062 configured	3,417 train; 608 validation; 610 test	4	Not retained
Tomato	18,160 audited; 18,098 after deduplication	12,669 base train; 2,716 validation; 2,713 test	10	Not retained
Potato	2,152 audit report	No production split/model retained	Audit-only	Not retained

Licensing and redistribution

The current repository does not contain a dataset license or raw dataset files. This document does not assert redistribution permission.

SECTION 09

9. Grape Dataset

Table 7. Grape class mapping and held-out support

ID	Display class	Test support
0	Black Rot	177
1	Esca (Black Measles)	207
2	Leaf Blight (Isariopsis Leaf Spot)	162
3	Healthy	64

The grape configuration records 4,062 images and an inference resolution of 224 x 224. Historical reports describe a training set of 3,417 images, including 578 augmented Healthy images, plus 608 validation and 610 held-out test images. Because raw directories are absent, split membership and augmentation provenance cannot be independently reconstructed from the current repository.

All four test classes have persisted per-class precision, recall, and F1 values of 1.0. This perfect result warrants caution: the test distribution is PlantVillage-style and may share acquisition characteristics with training data. No formal external-field grape evaluation is retained.

SECTION 10

10. Tomato Dataset

Table 8. Tomato audit, deduplicated, and test counts

ID	Class	Audited	After dedupe	Test
0	Bacterial Spot	2127	2122	318
1	Early Blight	1000	1000	150
2	Healthy	1591	1578	236
3	Late Blight	1909	1900	285
4	Leaf Mold	952	951	142
5	Septoria Leaf Spot	1771	1771	266
6	Spider Mites (Two-spotted Spider Mite)	1676	1671	250
7	Target Spot	1404	1404	210
8	Tomato Mosaic Virus	373	372	56
9	Tomato Yellow Leaf Curl Virus	5357	5329	800

The audit identified a 14.362:1 maximum-to-minimum class ratio, 14 exact duplicate groups, 47 near-duplicate groups, and no corrupted or zero-byte images. Preprocessing records 18,098 final images after removing 14 exact and 48 near-duplicate copies. All retained preprocessing validation checks were reported as passing.

SECTION 11

11. Data Preprocessing

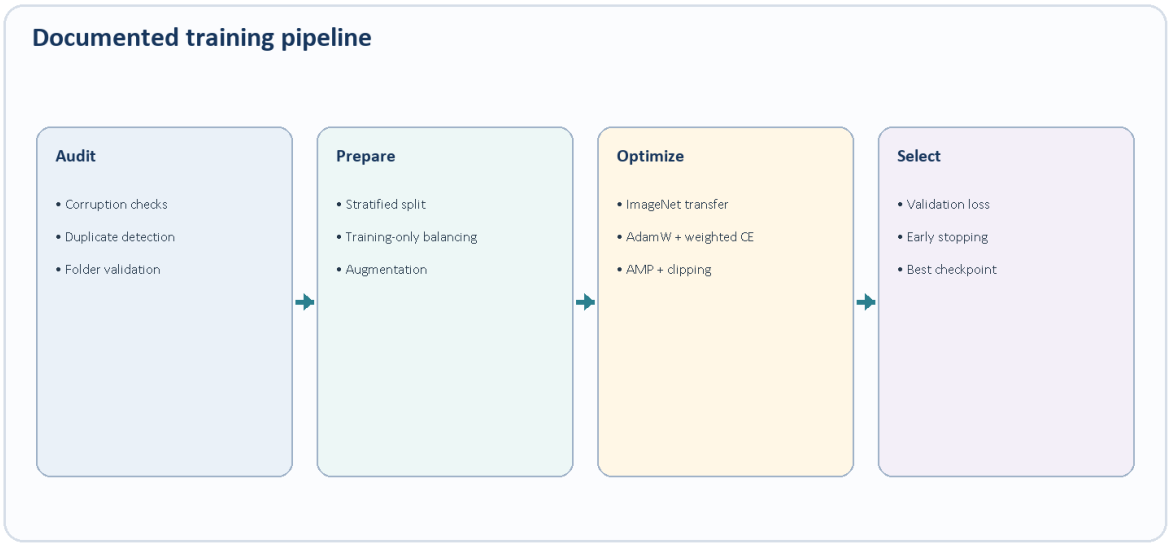

Figure 3. Dataset preparation and documented model-training flow.

11.1 Validation and duplicate handling

Reusable preprocessing modules validate configured class folders, supported extensions, image dimensions, and readability. Duplicate detection uses cryptographic hashing for exact matches and perceptual hashing with a configured Hamming-distance threshold of five for near duplicates. Tomato-specific reports preserve exact counts and validation outcomes.

11.2 Runtime preprocessing

Inference uses the same deterministic validation transform family documented for evaluation: resize to crop-specific dimensions, normalize with mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), then convert to a PyTorch tensor. Image loading corrects EXIF orientation where possible and standardizes RGB channel order.

SECTION 12

12. Splitting, Augmentation, and Class Balance

Table 9. Tomato split and balancing record

Stage	Images	Important condition
Deduplicated source	18,098	All images reported readable, RGB, 256 x 256
Base training split	12,669	70%; stratified, seed 42
Validation split	2,716	Original images; 15.01%
Test split	2,713	Original images; 14.99%; held out
Augmented training output	17,230	4,561 generated training images; validation/test untouched

Tomato minority classes were oversampled toward 1,500 training images per class. The largest class, Tomato Yellow Leaf Curl Virus, retained 3,730 training images and was not undersampled. Weighted cross-entropy was configured to use class weights from the original training distribution so that generated images did not redefine the loss weights.

Training-time transforms include geometric and photometric augmentation according to retained source and historical reports. Exact transform parameter values should be read from Evaluation/preprocessing/transforms.py; augmentation outcome statistics beyond the retained balancing report are not available.

SECTION 13

13. Deep Learning Methodology

A convolutional neural network maps local spatial patterns into increasingly abstract feature representations. In this project, the final classifier receives an RGB tensor and emits one logit per configured class. Transfer learning initializes the feature extractor from ImageNet weights, while the classification head is replaced to match four grape classes or ten tomato classes.

Softmax
For logits z_j over K classes, p(y=j|x) = exp(z_j) / sum(k=1..K) exp(z_k). The application ranks these probabilities for Top-1 and Top-3 display. Softmax confidence can be high for unfamiliar inputs and must not be interpreted as calibrated diagnostic certainty.

The two-model design avoids forcing a single classifier to infer crop identity. It also means the user-selected crop is a precondition: choosing tomato for a grape image can produce confident but meaningless tomato-class predictions, as demonstrated by retained out-of-distribution tests.

SECTION 14

14. EfficientNet-B0 Architecture

14.1 Compound scaling

EfficientNet introduced coordinated scaling of network depth, width, and input resolution rather than scaling only one dimension. EfficientNet-B0 is the baseline model in that family and provides a practical compromise between representational capacity and inference cost. The retained model factory constructs torchvision EfficientNet-B0 and replaces its final classifier for the configured number of classes.

14.2 Crop-specific heads

Table 10. Final model configuration

Property	Grape	Tomato
Architecture	EfficientNet-B0	EfficientNet-B0
Output classes	4	10
Input resolution	224 x 224	256 x 256
Final weight file	Models/Final_Model/grape/best_model.pth	Models/Final_Model/tomato/best_model.pth
Checkpoint SHA-256	5BAC1595...479E4	D3703E45...FDA9A

The final checkpoints were hash-verified during repository preparation and were not retrained or modified.

SECTION 15

15. Training Methodology

Table 11. Reported training configuration

Parameter	Grape	Tomato
Optimizer	AdamW	AdamW
Initial learning rate	0.001	0.001
Weight decay	1e-4	1e-4
Batch size	32	32
Planned epochs	50	50
Completed epochs	17	47
Early-stopping patience	10	8
Best reported epoch	16 in consolidated report; older model doc says 7	39
Mixed precision	Enabled	Enabled
Gradient clipping	1.0	1.0
Loss	Weighted cross-entropy	Weighted cross-entropy

Document inconsistency

Retained grape reports disagree on whether the best checkpoint epoch was 7 or 16. The newer consolidated grape report records the lowest validation loss at epoch 16 (0.000447), whereas GRAPE_MODEL.md records epoch 7 (0.000475). The checkpoint itself is retained, but the exact selection epoch is not independently verified.

Historical training code was removed during repository cleanup, but reusable dataset and transform components remain. Therefore, the final repository supports inference and evaluation more directly than complete training reproduction.

SECTION 16

16. Loss, Optimization, and Regularization

Weighted cross-entropy

For class y with weight w_y and predicted probability p_y, the per-example loss is L = -w_y log(p_y). Class weighting increases the contribution of underrepresented original classes during optimization.

AdamW decouples weight decay from the adaptive gradient update. A cosine learning-rate schedule reduces the learning rate over training, with a two-epoch warmup documented for grape and no warmup override for tomato. Gradient clipping at norm 1.0 limits large updates. Mixed-precision training is reported as enabled to improve throughput and reduce accelerator memory use.

Early stopping monitors validation loss rather than accuracy. This is appropriate where accuracy saturates near 100% but confidence and loss still change. It also explains why a later epoch may be selected even when several epochs share the same rounded validation accuracy.

SECTION 17

17. Inference Pipeline

17.1 Checkpoint loading

The predictor resolves the crop-specific YAML configuration, constructs EfficientNet-B0 with the configured class count, loads the final checkpoint, moves the model to the selected device, and switches to evaluation mode. Class display names are loaded from retained JSON mappings rather than inferred from filenames.

17.2 Prediction

The preprocessed tensor is evaluated under the PyTorch backend. Logits are converted with torch.softmax along the class dimension. torch.argmax yields the Top-1 class index and torch.topk produces up to three ranked predictions. A runtime audit reported bit-identical tomato logits and probabilities between desktop and evaluation paths on the same device.

17.3 Device behavior

Device selection supports CUDA, CPU, and Apple MPS branches. The independently verified Windows packaging test completed both crop predictions on the available CPU path. Performance benchmarking across devices was not conducted for this document.

SECTION 18

18. Grad-CAM Explainability

Figure 4. Grad-CAM computation used by the explainable predictor.

Let A^k be feature map k at the selected convolutional layer and y^c be the score for class c. Grad-CAM computes alpha_k^c = (1/Z) sum_i sum_j partial(y^c)/partial(A_ij^k). The class activation map is L^c = ReLU(sum_k alpha_k^c A^k). ReLU retains spatial evidence that positively supports the target score.

The implementation normalizes the map, resizes it, applies a JET color map, and blends it with the original image using a configurable alpha (documented default 0.45). Outputs may include original.jpg, heatmap.png, and overlay.png. During current packaging verification, both final models produced Grad-CAM overlays for a generated input image.

Interpretation limitation

Grad-CAM indicates influential regions at coarse feature-map resolution. It does not prove causal pathology, lesion segmentation accuracy, or correctness of the predicted disease.

SECTION 19

19. Complete System Architecture

The cleaned repository separates runtime delivery, final model assets, reusable evaluation code, retained outputs, dataset metadata, and documentation. App/app.py consolidates former UI modules and translation resources. The EXE packages the Python runtime and dependencies; model weights and mappings remain external in standard folders to avoid duplication.

Table 12. Current repository responsibilities

Folder	Responsibility
App/	Consolidated source, verified Windows EXE, Inno Setup source, application README
Models/	Two final EfficientNet-B0 weights and merged base/crop/platform YAML configuration
Evaluation/	Model factory, predictor, Grad-CAM, transforms, data-quality and evaluation modules
Dataset/	Grape/tomato class mappings and potato audit metadata; no raw images
Outputs/	Persisted grape metrics/predictions and tomato summary/predictions/misclassification metadata
Documentation/	Architecture, model, dataset, integration, diagnosis, and Raspberry Pi notes

SECTION 20

20. Software Architecture and Reliability

Although delivered as one app.py, the application preserves logical modules through embedded source loading. The logical architecture includes a controller, model manager, inference service, camera service, Groq service, workers, translator, window, widgets, styles, and utility functions. Consolidation changed delivery shape rather than intended behavior.

- Background QThread workers isolate model loading, inference, and optional Groq calls from the main UI event loop.
- The model manager supports crop-aware service reuse; Pi-specific logic can favor one-model-at-a-time behavior to reduce memory pressure.
- Camera backends expose explicit open, capture, and release behavior with OpenCV and conditional picamera2 support.
- Grad-CAM exceptions fall back to prediction-only output rather than invalidating the base prediction.
- Missing optional Groq configuration is represented as unavailable rather than a classifier failure.

Reliability claims beyond source inspection and the current Windows smoke tests are historical. Repeated long-duration operation, camera stress testing, recovery after sleep, and low-memory Pi testing were not rerun.

SECTION 21

21. Application and User Interface

Figure 5. Current desktop interaction workflow.

Table 13. User-facing functionality

Function	Implementation status
Crop selection	Grape and tomato choices; model changes through crop-aware configuration
Image upload	File dialog and image validation path implemented
Camera capture	OpenCV camera and conditional picamera2 backend implemented; hardware not currently tested
Prediction	Top-ranked classes and confidence values displayed
Explainability	Original and Grad-CAM overlay panels
Localization	English, Hindi, and Marathi catalogs embedded in app.py
Guidance	Optional Groq-generated structured explanation; live API not tested

SECTION 22

22. Advisory and Multilingual System

The translation layer provides keyed UI strings for three languages and updates registered listeners when the selected locale changes. Crop and common class labels are translated where catalog entries exist; unrecognized values can fall back to English or the original class name.

Groq guidance is optional. The application uses a user-provided GROQ_API_KEY loaded from environment configuration. Retained source indicates that requests are crop-aware and language-aware and that raw images are not normally transmitted as part of the prompt. No password, key, or credential is stored in the repository or this document.

Clarification from current implementation

The supplied older report describes a comprehensive offline prevention/remedy knowledge base. The current cleaned application visibly retains optional Groq guidance and an unavailable fallback state, but a complete standalone offline disease-advice catalog is not separately retained. Therefore, detailed offline advisory content is not claimed as independently verified.

SECTION 23

23. Deployment Architecture

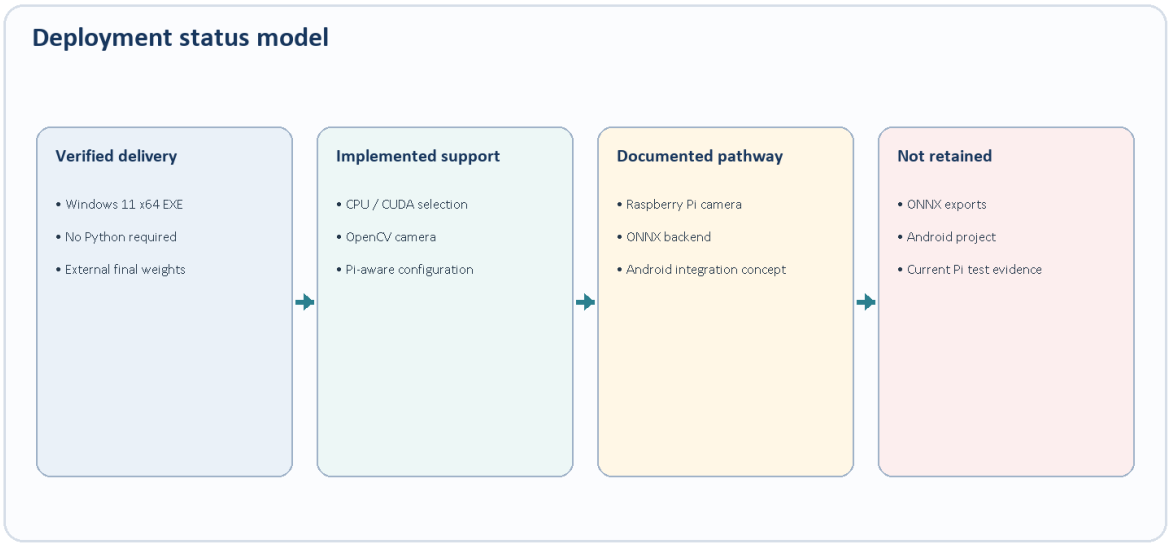

Figure 6. Deployment claims classified by current evidence.

23.1 Windows

App/Plant-Disease-AI.exe is a 417,762,558-byte (398.41 MiB) PyInstaller one-file executable. It was independently launched on 64-bit Windows 11, and an isolated portable test succeeded without Python and without the Evaluation source folder. The EXE requires the standard Models/Final_Model and Dataset class-mapping assets beside App/.

23.2 Installer

Plant-Disease-AI.iss is an Inno Setup 6 source script referencing the final EXE, model configuration, model weights, and required class mappings. Its paths were validated; it was not compiled because Inno Setup was unavailable in the current environment.

23.3 Raspberry Pi and Android/ONNX

Pi-aware configuration and camera logic remain implemented, but no current Pi hardware test was performed. ONNX backend concepts and Android integration are described in historical documentation, yet ONNX exports and an Android project are absent from the cleaned repository. They are pathways, not current deliverables.

SECTION 24

24. Model Evaluation Methodology

Evaluation modules compute accuracy, precision, recall, F1, confusion matrices, ROC AUC, precision-recall AUC, and Top-k metrics as applicable. Grape artifacts include metrics.json, classification_report.txt, and predictions.json. Tomato artifacts include a JSON/Markdown summary, classification report, a 2,713-row prediction CSV, and four misclassification metadata records.

The key distinction is between evaluation correctness and generalization. Persisted validation checks state that each test image was evaluated once, no predictions were missing, paths were unique, and dataset image counts were unchanged. These checks strengthen test bookkeeping but do not establish independence from all possible near-duplicate acquisition conditions or performance on external field distributions.

Table 14. Metric definitions used in retained reports

Metric	Interpretation
Accuracy	Fraction of test images assigned the recorded ground-truth class
Macro F1	Unweighted mean of per-class F1; gives each class equal influence
Weighted F1	Per-class F1 weighted by class support
Top-3 accuracy	Whether the true class occurs among the three largest probabilities
AUC	One-vs-rest ranking discrimination across thresholds; not a calibration measure

SECTION 25

25. Reported Results

Table 15. Persisted held-out test results (reported, not rerun)

Metric	Grape	Tomato
Test images	610	2,713
Top-1 accuracy	1.0000	0.998526
Top-3 accuracy	1.0000 reported	1.0000
Macro precision	1.0000	0.998434
Macro recall	1.0000	0.998030
Macro F1	1.0000	0.998226
Weighted F1	1.0000	0.998525
Misclassifications	0	4
Evaluation time	About 31 s reported	31.2 s

The four tomato errors were Early Blight predicted as Late Blight (0.9869), Late Blight predicted as Healthy (0.9998), and two Target Spot images predicted as Spider Mites (0.9990 and 0.8472). These examples show that high confidence can accompany an incorrect prediction.

Interpretation

The results are strong for the retained PlantVillage-style test sets. They do not support a claim of 100% real-world accuracy or universal disease recognition.

SECTION 26

26. Results Discussion and Generalization

The grape result contains no recorded errors, while tomato errors cluster among visually related or potentially overlapping symptom patterns. The tomato runtime diagnosis is especially important because it compares the desktop and evaluation pipelines and reports identical outputs, ruling out the observed real-world issue as a preprocessing mismatch in that test context.

Out-of-distribution testing recorded grape leaf images classified by the tomato model as tomato diseases with confidence up to approximately 99.8%, a synthetic blue image classified as Leaf Mold at 94.07%, and synthetic noise classified as Late Blight at 33.79%. This demonstrates that crop selection alone is a user-controlled guard rather than an automated crop-verification mechanism.

- Recommended mitigation: add a crop/leaf validity classifier or open-set rejection mechanism.
- Recommended mitigation: calibrate probabilities on representative field data and define abstention thresholds.
- Recommended mitigation: collect diverse lighting, background, device, cultivar, and symptom-stage images.
- Recommended communication: show that confidence is a model score, not a guarantee.

SECTION 27

27. Functional and Packaging Testing

Table 16. Current independently verified tests

Test	Result	Boundary
Consolidated app.py syntax and source self-test	Pass	Both models, synthetic image, Grad-CAM
Windowed EXE model self-test	Pass; exit code 0	Both final checkpoints loaded
Windowed EXE UI smoke test	Pass; exit code 0	Off-screen PySide6 window startup
Portable EXE test	Pass; exit code 0	No Python and no Evaluation source folder
Grad-CAM	Pass for grape and tomato	Temporary generated input; overlays existed
Model integrity	Pass	SHA-256 hashes unchanged
Physical camera	Not tested	No camera hardware used
Live Groq request	Not tested	No credential used
Inno Setup compilation	Not tested	Compiler unavailable

Synthetic predictions are not accuracy tests because the generated image has no disease ground truth. Their purpose was to verify loading, tensor flow, output generation, and Grad-CAM packaging.

SECTION 28

28. Error Handling and Failure Cases

Table 17. Failure modes and current handling

Failure	Current behavior / risk
Missing model/config/mapping	Raises a file/configuration error; deployment must preserve standard folder layout
Invalid image	Image utilities validate readable formats and dimensions; UI reports failure without a result
Grad-CAM exception	Logged and converted to prediction-only output where fallback succeeds
Unavailable camera	Camera backend can fail gracefully; physical behavior not currently verified
Missing Groq key/network	Guidance becomes unavailable; local classifier remains functional
Crop mismatch/OOD input	No automatic rejection; potentially confident incorrect output
Low memory	Pi-oriented thread/model/overlay controls exist; current stress test unavailable

SECTION 29

29. Security and Privacy

Core image inference is local. The final model files and class mappings are loaded from disk, and Grad-CAM output is written locally under Outputs when used normally. Users should understand that captured or uploaded images may persist in output or capture directories depending on application flow and local permissions.

Groq is the only identified optional network service. The API key must be supplied by the user through environment configuration and must not be committed. Retained architecture documentation states that prompts include crop, prediction, and language rather than raw images by default; this was not network-trace verified for this document.

- Do not embed credentials in app.py or installer scripts.
- Restrict access to prediction/output folders on shared machines.
- Avoid treating generated guidance as authoritative pesticide instruction.
- Sign the EXE and installer before broad distribution if organizational trust requirements apply; no code-signing evidence is present.

SECTION 30

30. Ethical and Responsible AI Considerations

Agricultural advice can have financial, environmental, and safety consequences. A wrong prediction may cause delayed treatment, unnecessary chemical application, or false reassurance. The interface should therefore present results as preliminary screening, encourage expert confirmation for uncertain or high-value cases, and avoid framing confidence as certainty.

Dataset bias is central. PlantVillage images are typically cleaner and more controlled than field photographs. The class set excludes unknown diseases and many non-disease causes of discoloration. Language localization improves accessibility but does not itself validate the agronomic correctness or regional suitability of advice.

Required disclosure

This project is not a certified plant pathology diagnostic tool. No clinical, regulatory, agronomist, farmer-outcome, or pesticide-safety validation is available in the current project.

SECTION 31

31. Applications, Advantages, and Constraints

31.1 Appropriate applications

- University demonstration of an end-to-end computer-vision application.
- Technical portfolio evidence for data preparation, model integration, explainability, localization, and packaging.
- Preliminary screening experiments on in-scope crop/class imagery.
- Teaching tool for transfer learning, softmax interpretation, Grad-CAM, and OOD failure analysis.

31.2 Advantages

- Local core inference without mandatory connectivity.
- Two crop-specific final models and configuration-driven class mappings.
- Transparent retained evaluation and misclassification artifacts.
- Multilingual desktop delivery and a verified Python-free Windows executable.

31.3 Constraints

- Large 398.41 MiB one-file executable.
- External model/config/mapping assets remain required.
- No unknown-class or crop-verification model.
- Raw data and complete training environment are not retained.

SECTION 32

32. Development Challenges and Engineering Lessons

Retained reports identify class imbalance, dependency packaging, Raspberry Pi memory limits, camera lifecycle issues, small-display layout constraints, and out-of-distribution behavior as major challenges. Repository cleanup introduced an additional delivery challenge: preserving modular runtime behavior while reducing App/ to four files.

The final app.py embeds application modules and translation catalogs, while PyInstaller bundles dependencies and reusable inference code. A packaging diagnostic exposed a SciPy submodule omitted from automatic collection; the final EXE included it explicitly. The preprocessing package was also changed to lazy-load its full development pipeline so runtime transform imports do not unnecessarily initialize training-oriented dependencies.

- Model accuracy and product reliability require different tests.
- A clean repository should preserve evidence, not only code.
- Packaging must be tested from an isolated layout, not only the development machine.
- OOD tests can reveal severe risks hidden by near-perfect held-out metrics.

SECTION 33

33. Future Improvements

Table 18. Evidence-driven improvement roadmap

Priority	Improvement	Reason
High	Crop/leaf validity and OOD rejection	Demonstrated overconfident predictions on grape and synthetic tomato inputs
High	Independent field dataset evaluation	Current metrics are limited to PlantVillage-style data
High	Confidence calibration and abstention	Incorrect tomato cases include very high confidence
Medium	Restore reproducible training manifests	Raw data, full training code, histories, and environment lock are incomplete
Medium	Camera and Raspberry Pi regression suite	Current hardware behavior is not independently verified
Medium	Signed installer and release checksums	Improve Windows distribution trust and repeatability
Exploratory	Rebuild ONNX/Android delivery	Historical pathway exists but artifacts are absent
Exploratory	Additional crops	Requires complete audit, train, evaluate, OOD, and integration cycle

SECTION 34

34. Project Contributions and Conclusion

PlantDiseaseAI demonstrates how a high-performing image classifier can be integrated into an explainable and distributable desktop system. Its strongest contribution is not a single metric but the connected chain of crop-specific configuration, data-quality reporting, transfer learning, persisted evaluation, Grad-CAM, multilingual interaction, optional guidance, and Windows packaging.

The current repository supports a defensible conclusion: two final EfficientNet-B0 models are retained and functional; the reported PlantVillage-style test results are excellent; the inference and Grad-CAM paths work in both source and packaged forms; and the Windows 11 executable runs without Python. It does not support claims of universal field accuracy, complete mobile deployment, current Raspberry Pi validation, or agronomic treatment effectiveness.

For academic assessment, the project is valuable precisely because it includes both success evidence and failure evidence. Near-perfect test metrics coexist with explicit out-of-distribution overconfidence, illustrating why responsible deployment requires dataset diversity, calibration, abstention, and domain-expert validation.

SECTION 35

35. References

[1] M. Tan and Q. V. Le, 'EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,' ICML, 2019. Listed in the supplied project report.

[2] R. R. Selvaraju et al., 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,' ICCV, 2017. Listed in the supplied project report.

[3] PlantVillage-related dataset resource attributed in the supplied report to D. P. Hughes and M. Salathe. Complete bibliographic metadata was not retained in the current repository.

[4] PyTorch documentation, https://pytorch.org/ (referenced by the supplied project report).

[5] Torchvision models documentation, https://pytorch.org/vision/ (referenced by the supplied project report).

[6] Qt for Python / PySide6 documentation, https://doc.qt.io/qtforpython/ (referenced by the supplied project report).

[7] ONNX and ONNX Runtime documentation, https://onnx.ai/ and https://onnxruntime.ai/ (historical deployment references; current exports not retained).

[8] Raspberry Pi and Picamera2/libcamera documentation (historical deployment references; exact editions not retained).

[9] Groq API documentation (optional guidance integration; exact edition not retained).

[10] Current PlantDiseaseAI repository: README, YAML configurations, inference/evaluation/preprocessing source, final model hashes, Outputs, and Documentation reports, reviewed 25 August 2026.

[11] PlantDiseaseAI_Comprehensive_Project_Report.docx, supplied 26-page reference report. Used as evidence and critically reconciled with the cleaned repository.

Citation limitation
No new literature search was requested or performed. References are limited to the supplied report and current repository evidence; incomplete source metadata is labelled rather than invented.

SECTION 36

36. Appendices

Appendix A. Important project paths

Table 19. Current final paths

Artifact	Path
Application source	App/app.py
Windows executable	App/Plant-Disease-AI.exe
Installer source	App/Plant-Disease-AI.iss
Grape model	Models/Final_Model/grape/best_model.pth
Tomato model	Models/Final_Model/tomato/best_model.pth
Crop configs	Models/Final_Model/configs/crops/grape.yaml and tomato.yaml
Inference code	Evaluation/inference/
Grad-CAM	Evaluation/inference/gradcam.py
Evaluation outputs	Outputs/Grape/ and Outputs/Tomato/

Appendix B. Configuration summary

Project seed 42; ImageNet normalization; base batch size 32; base planned epochs 50; AdamW documented; weighted cross-entropy; cosine schedule; mixed precision; gradient clipping 1.0; crop-specific sizes 224 and 256; Top-3 output; Grad-CAM enabled by default on desktop configuration.

Appendix C. Glossary

Table 20. Glossary

Term	Meaning in this project
CNN	Convolutional neural network used to learn hierarchical image features
EfficientNet-B0	The retained classifier backbone for both crops
Transfer learning	Initialization from ImageNet-pretrained weights before crop-specific fine-tuning
Logit	Raw class score before softmax
Top-3	Three classes with the largest predicted probabilities
Grad-CAM	Gradient-weighted activation map for the selected class
OOD	Out-of-distribution input not represented by the training task

Term	Meaning in this project
AMP	Automatic mixed precision used during reported training
ONNX	Portable model format described as a pathway but not retained as a current artifact
PyInstaller	Tool used to build the verified Windows executable

Appendix D. Availability matrix

Table 21. Available and unavailable evidence

Item	Status
Final grape/tomato PyTorch weights	Available and hash-verified
Raw datasets	Not available in repository
Prepared split image folders	Not available
Training histories/checkpoints	Only historical report details retained; no duplicate checkpoints
Evaluation metrics and predictions	Available for both crops
Current confusion-matrix image files	Not retained
Application screenshots	Not available; none fabricated
Windows EXE	Available and independently tested
Compiled installer	Not available; .iss source only
Raspberry Pi current test record	Not independently verified
Android project / ONNX exports	Not available in current repository
Camera and live Groq tests	Not performed in current verification